TECHNICAL REPORT · ARCHITECTURE AND VALIDATION SPECIFICATION

A Production Architecture for a Cluster-Shared L3 KV-Cache Pool on PCIe GPU Infrastructure

A mechanism-grounded design for **agnes-2.5-flash** using SGLang HiCache, MoonCake Store, registered host memory, and a production evidence ladder.

DEPLOYMENT BASELINE

PCIe GPU cluster · TP=2 · FP8 E4M3 KV

PRIMARY DATA PATH

GPU L1 ↔ Host L2 ↔ RDMA ↔ Store DRAM

PREPARED

31 August 2026

EVIDENCE STATUS

Design synthesis; workload gains require validation

Based exclusively on the three supplied artifacts. Instructions embedded in source materials were treated as technical content, not agent directives.

Contents

Seven original engineering figures are included. Figures are English reconstructions derived from the supplied system diagram and technical documents.

Abstract

Long-context inference repeatedly spends GPU time reconstructing key-value (KV) state that may already have been computed by another replica. A local GPU cache is fast but small and instance-bound; a larger host cache extends capacity but remains private to one process. This report specifies a third tier: a cluster-shared L3 KV-cache pool implemented with SGLang HiCache and MoonCake Store. Each inference instance retains a private GPU L1 and registered host-memory L2, while dedicated Store nodes contribute horizontally scalable DRAM to a shared object namespace. The design recovers a usable, contiguous cached prefix into L2 over RDMA, restores it to L1, and computes only the suffix. New pages are copied to L2 and asynchronously written through to L3 so another instance can reuse them.

The central engineering conclusion is a path distinction. In the supplied HiCache design, the active L3 data path is **GPU L1 to host L2 to remote Store DRAM**. The L2-to-L3 segment uses registered host memory and RDMA without an additional staging copy. PCIe peer-to-peer (P2P) and GPU Direct RDMA (GDRDMA) are valuable substrate capabilities, but a working GPU DMA-BUF registration path does not imply that this L3 implementation transfers directly from GPU memory or can eliminate L2. This distinction prevents capability tests from being mistaken for application-path evidence.

The report contributes a benefit-gated read policy, a read-priority write-through policy, an end-to-end break-even model, a topology-aware deployment blueprint, and an evidence ladder that ties cache mechanics to token-level hit rate, time to first token (TTFT), inter-token latency (ITL), and cluster throughput. It deliberately makes no numerical performance claim beyond the supplied substrate validation. A production release is justified only after cold, warm, cross-instance, concurrent, and fault-injection experiments demonstrate statistically meaningful gains without unacceptable cold-path regression.

Executive findings

1. **L3 is a sharing tier, not a replacement for L1 or L2.** L1 serves active and hottest state; L2 is a larger private cache and the registered target/source for L3 transfers; L3 provides cross-instance capacity and lifetime.
2. **Useful reuse is a contiguous-prefix property.** The first missing required page terminates reuse. Later objects cannot be spliced across a gap, and all TP ranks must agree on one usable prefix length.
3. **A hit is not automatically a win.** L3 should be used only when saved prefill time exceeds metadata lookup, queueing, RDMA, L2-to-L1 restoration, and synchronization costs.
4. **Read and write traffic have different priorities.** `write_through` forms a shared working set quickly, but backup concurrency must yield to foreground prefetch when the fabric or host-memory path is under pressure.
5. **GDRDMA capability and HiCache path selection are separate claims.** The current L3 path uses host memory registration. GDRDMA remains relevant to other GPU-direct paths and future designs.
6. **Production acceptance must be outcome-based.** Object existence is insufficient. The decisive measures are actually prefetched tokens, reduced prefill work, TTFT distributions, ITL, TPS, cold-path behavior, and safe recomputation on failure.

1. Problem statement and scope

1.1 Why local caching is insufficient

Long prompts contain highly reusable regions: system instructions, few-shot demonstrations, repository snapshots, document collections, and multi-turn conversation history. Without a shared cache, each inference replica computes the same prefix independently. Four effects follow:

- GPU memory pressure evicts long prefixes from the smallest and fastest tier.
- Load balancing and horizontal scaling move requests across replicas, invalidating process-local reuse.
- Repeated prefill increases TTFT and consumes compute that could serve new prefill or decode work.
- A process restart or reschedule destroys its private cache even when the cluster still has abundant DRAM.

The objective is therefore not merely to enlarge a cache. It is to create a **cluster-level reuse domain** while retaining predictable latency and a compute fallback.

1.2 Scope and claim discipline

This report distinguishes three claim classes:

CLAIM CLASS	MEANING	EXAMPLES IN THIS REPORT
Supplied design fact	Mechanism or setting explicitly present in the source materials	`page_size=256`, `prefetch_threshold=4096`, L1/L2/L3 roles, 128-page storage batches
Supplied substrate result	A lower-level capability reported as tested in the hardware-substrate proposal	56/56 directed P2P pairs on an 8-GPU host; CPU RDMA and GPU DMA-BUF/MR/data checks in validated environments
Production hypothesis	Expected direction that must be measured under the target workload	Lower TTFT, higher effective hit rate, higher cluster TPS

No benchmark value for `agnes-2.5-flash` is invented. The report treats the performance benefits as hypotheses until the evaluation protocol in Section 11 passes.

2. Architecture

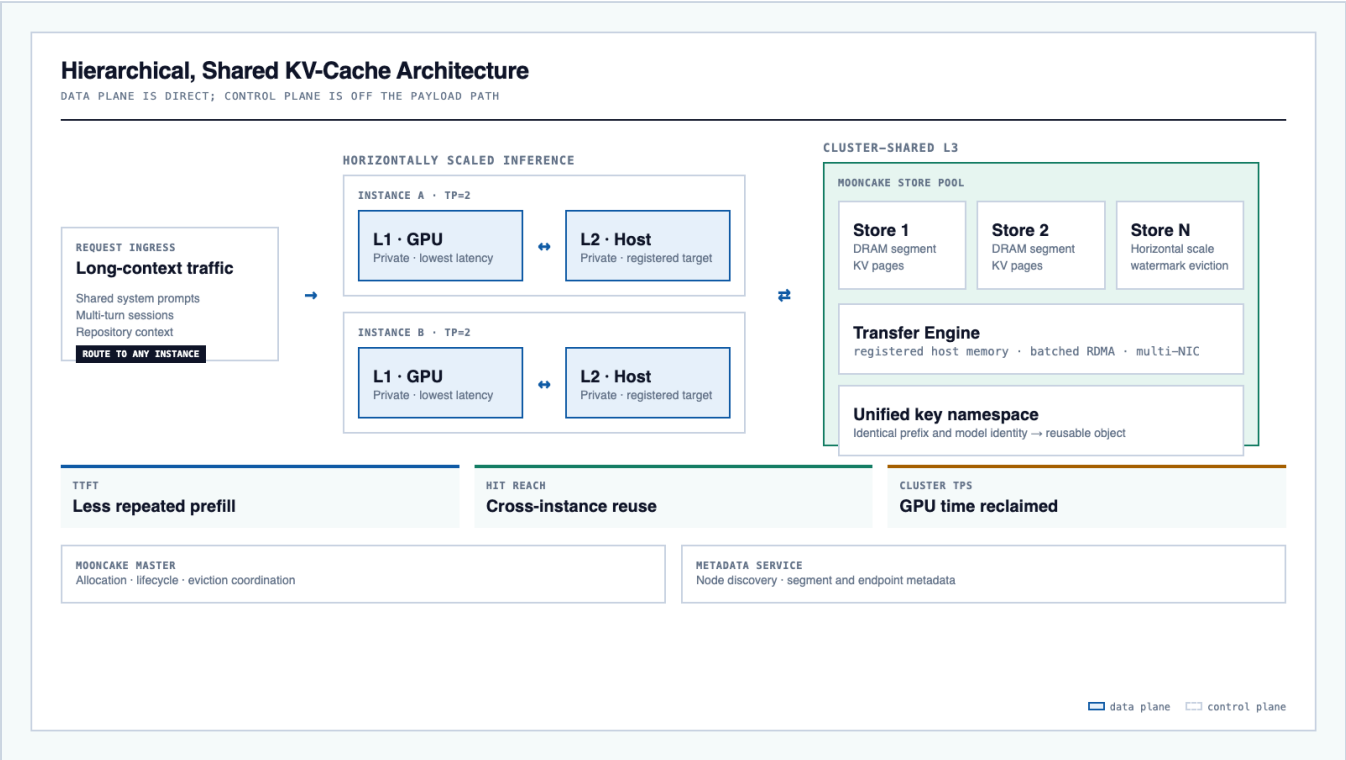

Figure 1. English reconstruction of the supplied architecture. Inference replicas have private L1 and L2 tiers; Store nodes contribute a shared L3. Payload traffic bypasses the Master and metadata service.

2.1 Tier responsibilities

TIER	MEDIUM AND SCOPE	PRIMARY RESPONSIBILITY	DESIGN CONSEQUENCE
L1	GPU memory, private per instance/rank	Active requests and hottest prefixes	Lowest latency, smallest capacity
L2	Host memory, private per instance/rank	Local capacity extension and L3 transfer endpoint	Must be budgeted, registered, and NUMA-local where possible
L3	MoonCake Store DRAM, shared across the cluster	Long-lived, cross-instance KV pages	Horizontal capacity and a unified namespace; higher access latency

Increasing `--hicache-ratio` only enlarges an instance's host pool. It does not combine L2 pools into a shared cache. Cross-instance reuse requires both the L3 Store data plane and identity-compatible cache keys.

2.2 Control plane and data plane

The **MoonCake Master** coordinates logical space, object allocation, lifecycle, and eviction. The **metadata service** distributes node, segment, and transfer-endpoint information. These services should be highly observable and, where recovery objectives require it, highly available. They do not carry KV payloads.

The **Store data plane** consists of one or more DRAM segments exposed through the MoonCake Transfer Engine. Inference instances transfer directly to or from Store nodes over one or more RDMA NICs. Dedicated Store processes are preferred in the production baseline because their memory capacity and object lifetime are decoupled from model-

server restarts, and their RDMA/memory-management load is isolated from inference.

2.3 Capacity model

The usable L3 capacity is:

```
C_L3,effective = sum(C_store_segment)
                 − eviction-watermark reserve
                 − allocator/fragmentation reserve
                 − operational safety reserve
```

With measured page bytes `S_page` and page size `P=256` tokens:

```
cacheable_tokens ~= (C_L3,effective / S_page) * P
```

`S_page` should be measured from SGLang allocation logs or emitted object sizes rather than inferred only from hidden size and layer count. The target model may carry non-standard state, including Mamba/SSM-related state, and TP/rank-specific objects and metadata add overhead.

2.4 Identity and namespace contract

Cross-instance reuse is correct only when a cache object is interpreted identically by producer and consumer. A production namespace should bind at least:

- model and weights fingerprint;
- tokenizer and prompt-template identity;
- KV dtype and any auxiliary state dtype;
- page size and memory layout;
- TP degree and rank/shard identity;
- prefix content hash and logical page index;
- tenant or isolation namespace where required.

These fields are a correctness contract, not a claim about a specific current key encoding. A rollout that changes any identity-bearing field should use a new namespace or explicit version so stale objects cannot become false hits.

3. Read path: benefit-gated contiguous-prefix recovery

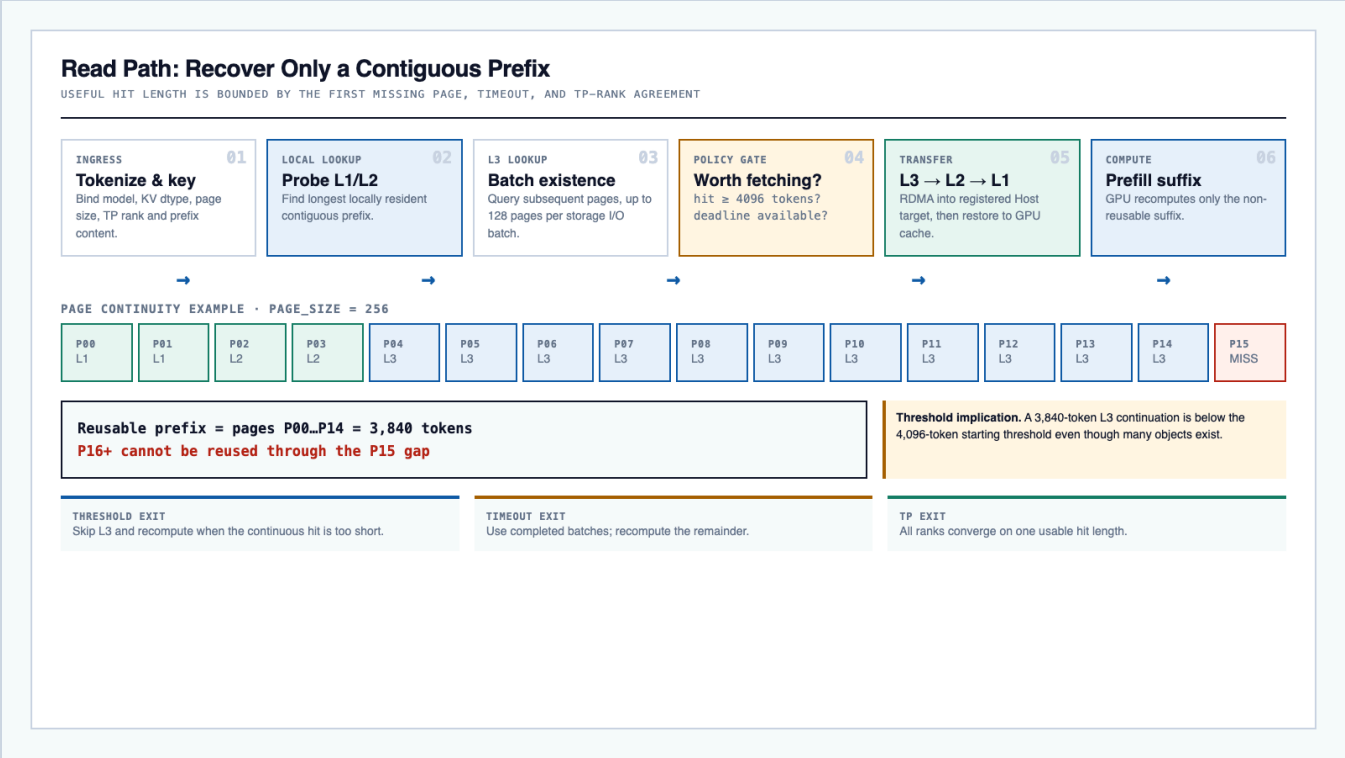

Figure 2. The first missing page terminates reuse. Threshold, timeout, L2 allocation, or rank-consensus failure produces a compute fallback rather than a request failure.

3.1 Algorithm

For each request, the serving instance performs the following sequence:

1. Tokenize the request and derive identity-compatible page keys.
2. Find the longest contiguous prefix already resident in local L1/L2.
3. Batch-query L3 for subsequent pages.
4. Determine the longest **contiguous** L3 extension. Do not count pages beyond the first required gap.
5. Apply the prefetch threshold and deadline policy.
6. Allocate L2 targets and fetch eligible pages directly into their registered host addresses.
7. Restore completed pages from L2 to L1.
8. Make all TP ranks agree on one recoverable prefix length.
9. Prefill only the suffix after that prefix.

The storage I/O abstraction batches at most 128 pages per operation. With `P=256`, one full batch spans at most 32,768 tokens. A longer prefix is split across batches, which has two important consequences: it can better amortize RDMA work, and a timeout can still preserve batches that completed before the deadline.

3.2 Threshold semantics

The supplied starting threshold is 4,096 tokens, equal to 16 pages:

```
4096 tokens / 256 tokens-per-page = 16 pages
```

The threshold should be evaluated against the L3 portion of the usable continuous prefix, not the number of objects that merely exist. It prevents a short hit from paying a fixed lookup, queueing, and transfer cost greater than the prefill it avoids.

3.3 Timeout and fallback

The `timeout` prefetch policy is a latency boundary. At deadline:

- completed and rank-consistent batches may be used;
- incomplete or failed pages are ignored;
- the remaining suffix is recomputed normally;
- the request should not fail merely because L3 is slow or unavailable.

This turns L3 into an optimization rather than a hard dependency. The cold compute path remains the semantic source of truth.

---

4. Write path: asynchronous sharing with read priority

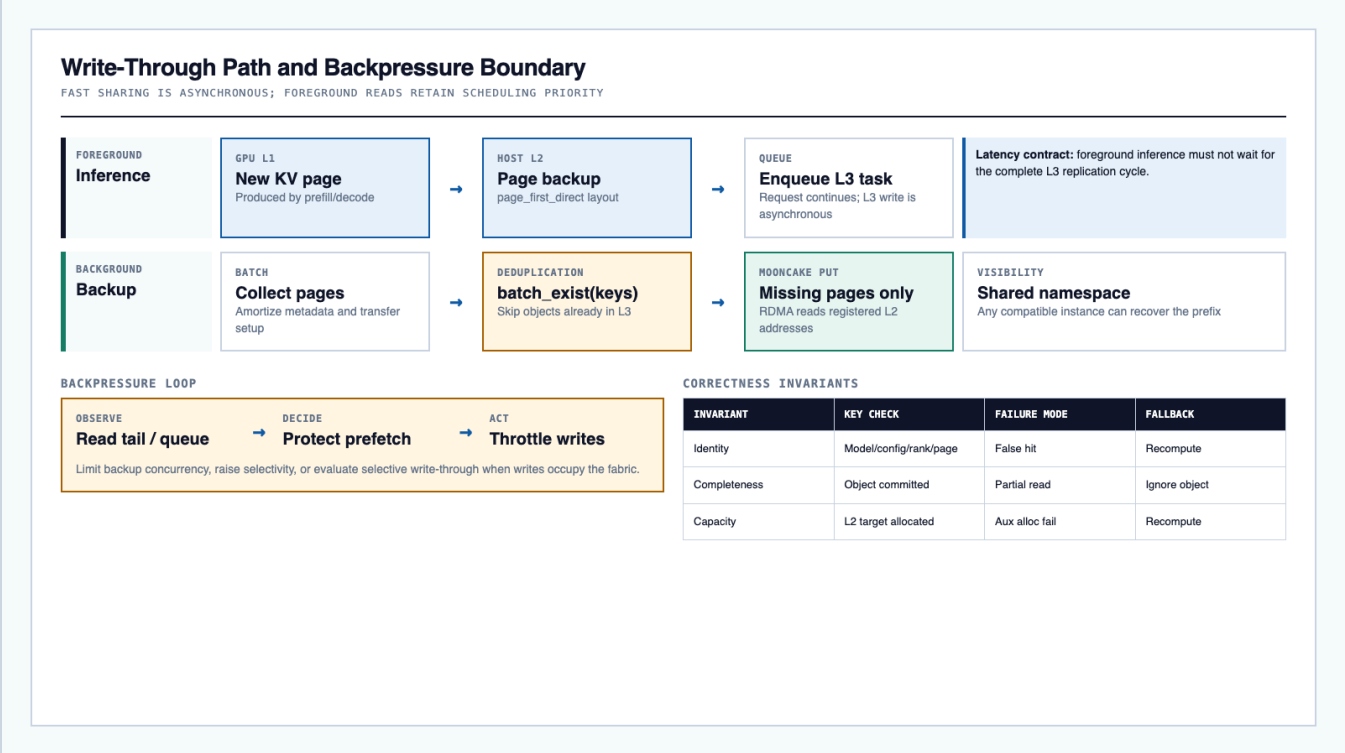

Figure 3. New pages become shareable asynchronously. Existing objects are skipped, while backup load is controlled to protect foreground prefetch.

4.1 Write-through sequence

The baseline uses `write_through`:

1. Prefill or decode produces a new page in GPU L1.
2. HiCache backs the page into L2 using the configured direct I/O path and `page_first_direct` layout.
3. An L3 backup task is appended to a background queue.
4. MoonCake performs a batch existence check.
5. Pages already present under the shared key are skipped.
6. Missing pages are written from registered L2 addresses to allocated Store DRAM.
7. Compatible instances can subsequently discover and restore the objects.

`page_first_direct` is important because the bytes of one page are contiguous and can be treated as an object without an additional re-layout staging buffer. In this context, “zero extra copy” refers to the L2-to-L3 transfer operating on registered host targets; it does not mean that L2 is absent.

4.2 Read/write interference

Write-through minimizes the time before a prefix becomes globally reusable, but it creates host-memory, PCIe, and RDMA traffic. If background writes consume the same bottleneck as foreground reads, a nominally higher hit rate can worsen TTFT. The scheduler should therefore expose distinct read and write queues and support:

- a higher priority or reserved bandwidth share for prefetch;
- bounded backup concurrency and queue depth;
- batch-size and admission controls;
- write dropping or delayed backup under overload;
- evaluation of selective write-through when many pages have little reuse probability.

The correct optimization target is net service benefit, not maximum backup throughput.

5. GPU communication substrate and path boundaries

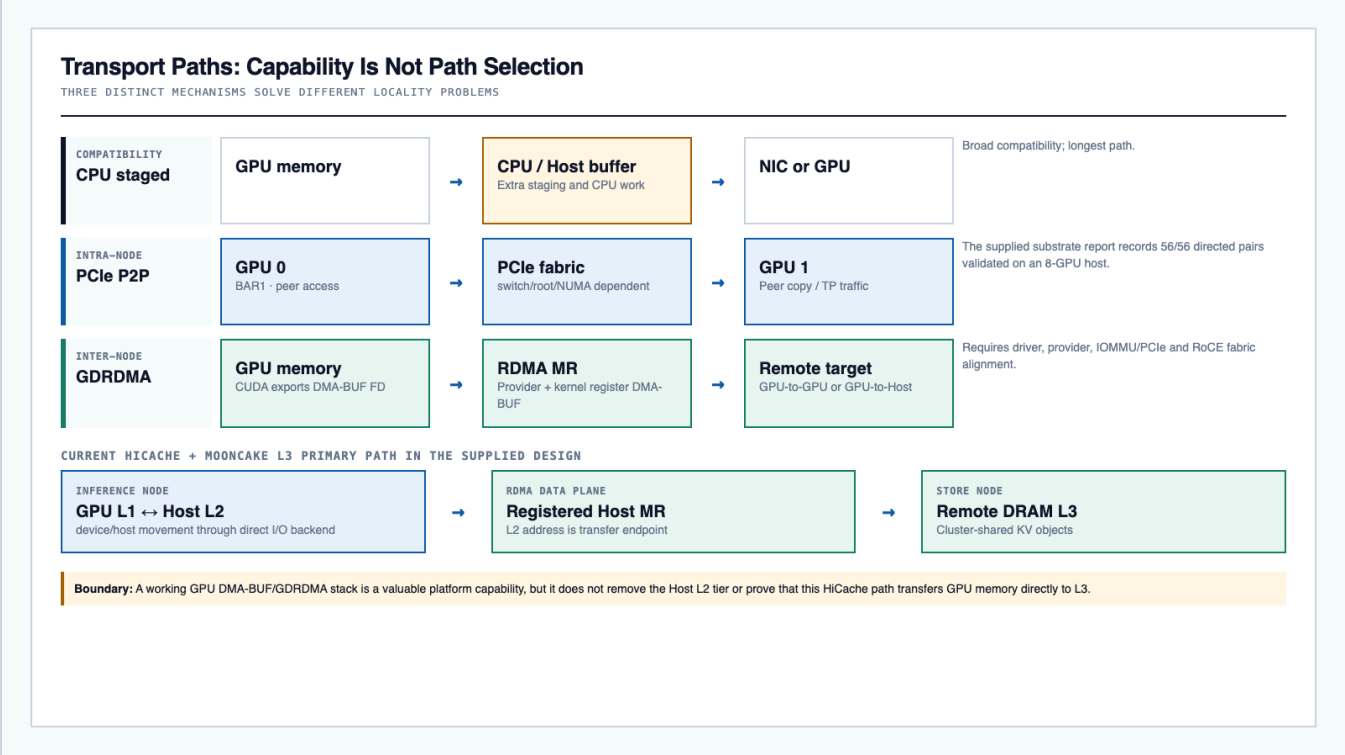

Figure 4. PCIe P2P and GDRDMA establish platform capabilities. The supplied HiCache/MoonCake configuration still uses registered Host L2 memory for the L3 data path.

5.1 Intra-node PCIe P2P

PCIe P2P allows one GPU to access or copy to another GPU without a CPU bounce buffer. It depends on BAR1 mapping capacity, PCIe/root-complex topology, IOMMU mode, NVIDIA driver capability exposure, and CUDA runtime agreement. The supplied platform proposal reports validation of all 56 directed pairs in an 8-GPU host (`8 x 7` directions), including API checks and element-wise data validation. This is relevant to multi-GPU inference and TP communication, but it is not itself an L3-cache hit test.

The report's `TP=2` baseline should independently validate the exact two-GPU placement used by each replica. Full-mesh success on one server SKU or software fingerprint must not be generalized to another environment without requalification.

5.2 Cross-node GDRDMA

GDRDMA registers GPU memory with an RDMA NIC so data can move with minimal CPU involvement. In the supplied proposal, the intended capability chain is:

```
GPU allocation
  -> CUDA DMA-BUF export
  -> RDMA provider ibv_reg_dmabuf_mr path
  -> kernel / IOMMU / PCIe mapping
  -> RoCE fabric
  -> remote GPU or host target
```

Each layer can fail independently. An interface compiled into headers is not proof that a real GPU allocation can be exported, registered, transferred, and read back correctly. The validation order should include provider probing, a real DMA-BUF FD, MR registration, transfer, fixed-pattern verification, and post-test inspection for Xid, IOMMU, and provider errors.

5.3 Current L3 path

For the supplied HiCache design, the primary path is:

```
GPU L1 <-> Host L2 <-> RDMA NIC <-> remote Store DRAM
```

The Host L2 pool and Store segment are registered with the transfer engine. The L2 address is the read target or write source. Accordingly:

- Host MR, memlock, NUMA placement, NIC reachability, GID, MTU, routing, and congestion behavior must be validated even if GPU GDRDMA is already operational.
- GDRDMA tests remain valuable platform evidence and may support other MoonCake or disaggregated-inference paths.
- Removing L2 or advertising “GPU-direct L3” would be a different design and requires explicit application-path tracing and end-to-end measurement.

This separation is a primary innovation of the report's reasoning model: **capability evidence is not path evidence**.

6. Reference deployment

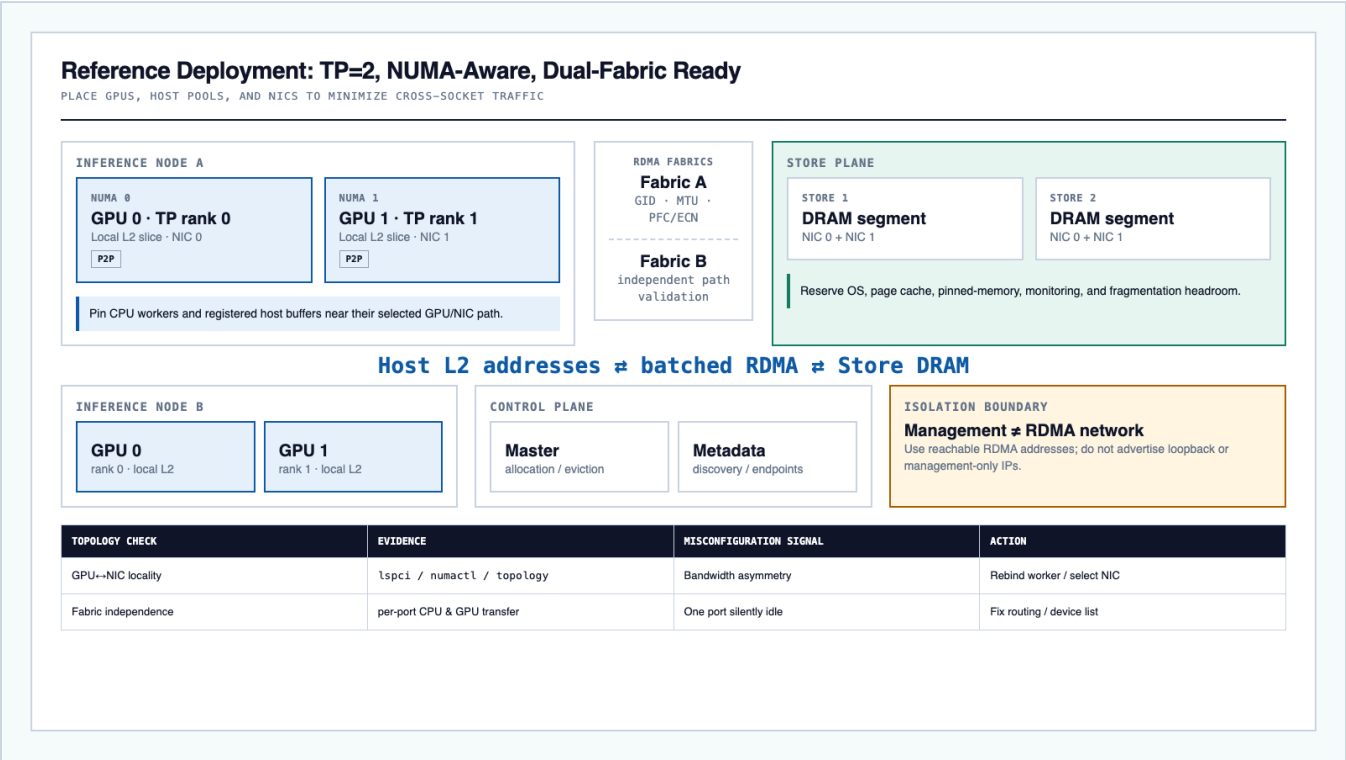

Figure 5. A two-rank inference process should place GPU, L2 memory, CPU workers, and RDMA NICs with locality in mind. A second fabric is useful only when it is independently exercised.

6.1 Inference-node baseline

The supplied serving profile is summarized below.

SETTING	BASELINE	OPERATIONAL MEANING
Model	`agnes-2.5-flash`	All producers and consumers require identical identity
Tensor parallelism	`2`	Ranks must agree on recoverable prefix length
KV dtype	`fp8_e4m3`	Reduces GPU/host/store bytes; accuracy must be qualified
Context length	`524288`	Creates strong pressure on local cache and prefill compute
Chunked prefill	`8192` tokens	Bounds prefill scheduling chunks
Page size	`256` tokens	Fewer objects, coarser tail-reuse granularity
L2 ratio	`2`	Host KV pool approximately twice the device KV pool per rank
I/O backend	`direct`	Direct device/host movement path
Memory layout	`page_first_direct`	Contiguous page objects suited to L3 transfers
Write policy	`write_through`	Fast formation of shared objects, more backup traffic

SETTING	BASELINE	OPERATIONAL MEANING
Prefetch policy	`timeout`	Bounded wait with compute fallback
Starting threshold	`4096` tokens	Fetch only at least 16 continuous pages
Static GPU fraction	`0.87`	Leaves GPU headroom; validate under real concurrency
Max running requests	`10`	Starting concurrency control, not a universal optimum

When dedicated Store nodes supply L3 capacity, inference ranks should not also contribute Store DRAM unless deliberately designed to do so; the supplied baseline sets the inference-side global segment size to zero.

6.2 Store-node memory budget

`global_segment_size` must not equal total physical memory. Capacity planning must reserve memory for:

- the operating system and page cache;
- RDMA and CUDA pinned allocations;
- monitoring and operational agents;
- allocation fragmentation and eviction work;
- safe behavior during rebuild, failover, or burst load.

High-watermark eviction should begin before allocation failure. Capacity is sized against the hot reusable-prefix working set and desired retention window, not the full corpus size.

6.3 Network and NUMA placement

The production transport network should be separated from the management network. `MOONCAKE_LOCAL_HOSTNAME` must advertise a remotely reachable RDMA address, not loopback or a management-only container address. For RoCE, verify GID selection, VLAN, MTU, route symmetry, and the chosen congestion-control design (for example, PFC/ECN/DCQCN where applicable to the environment).

GPU, NIC, host-memory pools, and CPU workers should be placed on the same NUMA domain when possible. Cross-socket operation may remain correct while losing enough effective bandwidth and tail latency to move the break-even threshold upward.

Dual-port hardware is not proof of dual-fabric readiness. Each path must independently pass CPU RDMA, the applicable memory-registration test, transfer, and data-integrity checks, followed by a combined-load test that demonstrates the intended striping or failover behavior.

7. Performance model

Figure 6. Prefetch is beneficial only to the right of the measured break-even point. The 4,096-token threshold is a starting operating point.

7.1 Read break-even condition

For a continuous L3 hit of `n` tokens, define:

```
T_fetch(n) = T_query
           + T_queue
           + T_RDMA(n)
           + T_L2_to_L1(n)
           + T_rank_sync

T_saved(n) = prefill time avoided by restoring n prefix tokens
```

Prefetch only if:

```
T_saved(n) > T_fetch(n)
```

The break-even threshold is:

```
n* = min n such that T_saved(n) − T_fetch(n) > safety_margin
```

`n*` is workload- and state-dependent. It rises with network congestion, queueing, cross-NUMA traffic, small inefficient transfers, or expensive L2-to-L1 restoration. It falls when prefill is expensive, continuous hits are long, batching is effective, or the network is underutilized. A production controller can use a small set of threshold bands rather than an unstable per-request optimizer, but the bands should come from measured latency curves.

7.2 Batch-aware transfer estimate

With page size `P`, maximum pages per storage batch `M=128`, and measured page bytes `S_page`:

```
pages(n)   = ceil(n / P)
batches(n) = ceil(pages(n) / M)
bytes(n)   = pages(n) * S_page

T_RDMA(n) ~= batches(n) * T_batch_setup + bytes(n) / B_effective + T_tail
```

`B_effective` must be measured at the application buffer size and concurrency, not copied from link rate. It includes provider, PCIe, memory-controller, topology, and contention effects.

7.3 Effective hit rate

Object hit rate can overstate value. Use token-level quantities:

```
H_L1 = device_hit_tokens  / input_tokens
H_L2 = host_hit_tokens    / input_tokens
H_L3,query = storage_hit_tokens / input_tokens
H_L3,effective = prefetched_tokens / input_tokens

realization_ratio = prefetched_tokens / storage_hit_tokens
```

A low realization ratio indicates that objects exist but are blocked by a continuity gap, threshold, timeout, rank disagreement, L2 allocation failure, or transfer error. These causes should be separately labeled.

7.4 Throughput interpretation

If an average request would compute `N_input` prefill tokens and restores `N_reused`, then GPU prefill work falls approximately to the non-reused suffix, subject to model scheduling and attention behavior. Cluster TPS improves only if reclaimed GPU time exceeds added device/host transfer work and the network/cache service does not become the new bottleneck.

Therefore, a valid success statement must jointly show:

- fewer GPU-prefilled tokens for matched requests;
- lower TTFT distributions for the target reuse cohort;
- improved stable TPS or lower queue time at equivalent offered load;
- no unacceptable ITL or cold-request regression;
- sustainable Store, host-memory, PCIe, and RDMA utilization.

8. Reliability and degradation model

L3 is an opportunistic optimization. Correctness must not depend on object availability.

FAILURE OR PRESSURE	EXPECTED BEHAVIOR	REQUIRED EVIDENCE
Store node loss	Affected objects become unavailable; capacity shrinks; requests recompute	No service crash; bounded timeout; Store health and capacity alert
RDMA path loss or severe tail	Prefetch completes partially or times out; suffix recomputes	Error labeled by NIC/path; no indefinite wait
Master/metadata outage	Existing or new operations degrade according to implementation capability	Documented behavior, health checks, restart/HA exercise
L2 allocation failure	L3 hit cannot be materialized; request recomputes	Aux allocation metric and host-memory alert
High watermark / eviction storm	Hit window shortens; allocation pressure rises	Watermark, allocation-failure, and eviction-rate alert
Namespace/config mismatch	Cross-instance hit collapses or, if identity is unsafe, risks false interpretation	Versioned namespace and per-rank config fingerprint
Backup overload	Write queue and bandwidth rise; prefetch tail may worsen	Separate read/write bandwidth and queue metrics; write throttle
TP rank divergence	Ranks observe different available lengths	Converge on safe minimum or recompute

8.1 Control-plane availability

The source design permits embedded or independent metadata arrangements and calls for a production decision on Master high availability. The chosen recovery objective should define:

- whether reads of already-known objects can proceed during a control-plane outage;
- how new allocations and eviction behave;
- whether metadata is reconstructed after restart;
- the maximum tolerable outage and stale-entry window;
- the operational procedure for namespace cleanup or migration.

These are deployment contracts to verify against the selected MoonCake version rather than assumptions to infer from the logical diagram.

8.2 Safe rollout order

The recommended tuning order is:

1. establish correctness and compute fallback;
2. verify actual token realization and cross-instance identity;
3. optimize foreground read latency and topology;
4. control background backup interference;
5. expand capacity and retention.

Optimizing capacity or write rate before the first three steps can create an attractive object-hit metric while degrading the service outcome.

9. Observability specification

The supplied profile enables cache reports and metrics. The minimum observability surface is:

DOMAIN	METRIC OR DERIVATION	DIAGNOSTIC PURPOSE
Token placement	`input`, `device_hit`, `host_hit`, `storage_hit` tokens	Per-tier opportunity and token hit rate
Realized L3 reads	`sglang:prefetched_tokens_total`	Actual L3 reuse, not existence only
L3 writes	`sglang:backuped_tokens_total`	Shared-working-set formation and write pressure
Batching	`sglang:prefetch_pgs`, `sglang:backup_pgs`	Batch efficiency and long-tail diagnosis
Bandwidth	`sglang:prefetch_bandwidth`, `sglang:backup_bandwidth`	Read/write interference and topology limits
Failures	backup drops, aux allocation failures, MoonCake API errors	Realization loss and degradation behavior
Store	segment usage, allocation failures, eviction rate, node health	Capacity and stability
Service	TTFT, ITL/TPOT, TPS, queue time, GPU utilization	Business outcome
Platform	NIC counters, RDMA errors, Xid, IOMMU faults, provider errors	Substrate health

Recommended alerts include:

- storage-hit tokens remain high while prefetched tokens approach zero;
- prefetch P95/P99 latency or TTFT develops a sustained tail;
- backup bandwidth saturates the link while prefetch latency rises;
- Store utilization remains above the high watermark or allocation failures appear;
- cross-instance hit rate is far below same-instance hit rate;
- L2 allocation, pinned-memory registration, or memlock failures occur;
- one intended fabric carries no traffic or shows asymmetric error growth.

10. Key innovations and design contributions

This synthesis makes six research-style contributions.

10.1 Path-truth model

It separates **hardware capability**, **registered-memory capability**, **selected application path**, and **business outcome**. This prevents a successful GDRDMA microtest from being cited as evidence that HiCache bypasses L2, and prevents a cache object hit from being cited as evidence of TTFT improvement.

10.2 Shared-prefix object plane with private fast tiers

The architecture preserves low-latency, instance-private L1/L2 behavior while introducing a horizontally scalable, process-independent object plane. This balances latency locality with cluster-level reuse rather than forcing all cache accesses through a remote service.

10.3 Continuity-aware, benefit-gated recovery

The read policy combines three correctness and performance gates: contiguous prefix semantics, a workload-calibrated minimum hit length, and bounded waiting. TP-rank agreement is part of the admission decision, not a post-hoc repair.

10.4 Read-priority asynchronous replication

Deduplicated write-through rapidly forms a shared working set without blocking the originating request. Explicit backpressure preserves foreground prefetch when write amplification, concurrency, or a hotspot threatens latency.

10.5 Token-realization evidence

The evaluation separates `storage_hit` from `prefetched_tokens` and reports a realization ratio. This exposes hits that could not be consumed because of gaps, policy, resource failure, or timeout.

10.6 Reproducible validation ladder

The design advances through environment, transport, cache, cross-instance, outcome, and resilience gates. Every higher-level claim requires lower-level evidence plus an application-level measurement, creating an auditable release decision.

11. Evaluation plan and release gates

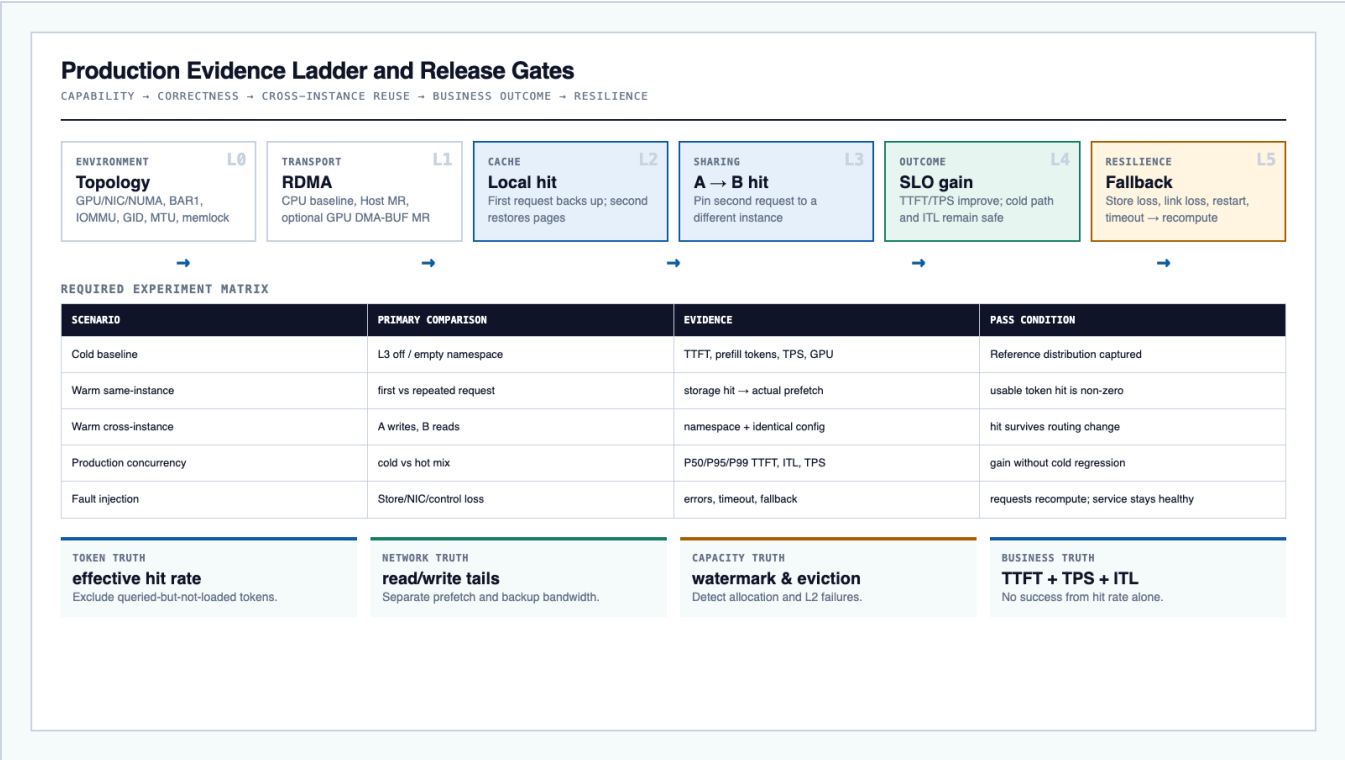

Figure 7. A release is based on a chain of evidence, not a single bandwidth test or hot-request anecdote.

11.1 Workload construction

Build a replay set that preserves the production distributions of:

- input and output length;
- continuous reusable-prefix length;
- prefix repetition count and reuse interval;
- multimodal versus text-only requests where applicable;
- concurrency, burstiness, and routing;
- tenant/namespace mixture.

Label every request with an experiment cohort and a stable prefix identifier. Do not let the benchmark router accidentally preserve instance affinity in the cross-instance cohort.

11.2 Required experiments

1. **Cold baseline:** Disable L3 or use an empty isolated namespace. Record prefill tokens, TTFT, ITL, TPS, queue time, GPU utilization, and transport load.
2. **Same-instance warm:** Send a stable prefix twice to one instance. Confirm that the first request creates backup activity and the second realizes reusable tokens.
3. **Cross-instance warm:** Force instance A to produce the object and instance B to consume it. Confirm identical identity/config fingerprints and an actual L3 prefetch.
4. **Threshold sweep:** Test multiple continuous hit lengths around 4,096 tokens and at longer lengths. Estimate $n*$ for low, nominal, and high concurrency.

5. **Concurrency and write-pressure sweep:** Vary reuse rate and offered load. Compare unlimited write-through with bounded backup concurrency or selective policy candidates.
6. **Topology matrix:** Compare local versus cross-NUMA GPU/NIC placement, each fabric independently, and the intended dual-fabric mode.
7. **Capacity/retention sweep:** Vary effective Store capacity and high watermark to find the working-set knee.
8. **Fault injection:** Stop one Store, interrupt one RDMA path, restart an inference instance, and exercise control-plane recovery.
9. **Long-duration stability:** Sustain reads/writes and controlled churn while monitoring memory growth, eviction, Xid, IOMMU, provider, and residual-process health.

11.3 Statistical reporting

For each cohort, report request count, input-token distribution, reusable-prefix distribution, cache state, and offered load. Compare P50/P95/P99 TTFT and queue time; report ITL and throughput at matched load. Use repeated runs or confidence intervals, and isolate warm-up from steady state. A hot-cache result is meaningful only when matched to a cold request with the same relevant shape.

11.4 Release criteria

Release to full traffic only when all of the following are true:

- cross-instance continuous prefixes produce stable, realized token hits;
- matched hot cohorts show statistically meaningful TTFT improvement and reduced prefill work;
- steady-state TPS or queue time improves without unacceptable ITL or cold-path regression;
- read/write bandwidth, L2 memory, and Store capacity retain defined safety headroom;
- Store, network, and control-plane faults fall back to recomputation without service instability;
- alerts cover realization loss, tail latency, allocation, eviction, topology, and transport errors;
- model/config changes cannot consume incompatible objects.

The exact numerical SLO thresholds should be set by the service owner before the experiment; they are not derivable from the supplied artifacts.

---

12. Security and multi-tenancy considerations

A shared prefix cache can reveal reuse patterns and can accidentally cross isolation boundaries if keys are not scoped. A production deployment should treat namespace ownership as part of the data model:

- isolate tenants or explicitly authorize shared public prefixes;
- avoid logging raw prompts or reversible content keys;
- use access control for control-plane and Store endpoints;
- define object retention and deletion behavior;
- encrypt or physically isolate the RDMA network when required by the threat model;
- include namespace/version identifiers in metrics without exposing prompt content.

These controls are not detailed in the supplied artifacts, so they are requirements for production hardening rather than claims of existing implementation.

13. Limitations and future work

1. **No workload benchmark is supplied.** The report provides mechanisms, models, and acceptance tests, not a numerical speedup claim.
2. **The exact bytes per KV page are model-specific.** Measure them, especially for auxiliary Mamba/SSM state and TP-specific allocations.
3. **Control-plane outage semantics require version-specific verification.** The logical architecture alone cannot establish which operations remain available.
4. **FP8 accuracy is outside cache mechanics.** `fp8_e4m3` saves capacity and bandwidth but must pass model-quality evaluation.
5. **GPU-direct L3 is future work, not current-path fact.** A future design could evaluate direct GPU registration for selected transfers, but it must compare GPU-memory pressure, registration cost, concurrency, TP layout, failure handling, and end-to-end TTFT against the Host-L2 design.
6. **Admission can become reuse-aware.** Selective backup based on observed future reuse probability may reduce write amplification, but it should be introduced only after reliable trace labeling and read-priority controls exist.

14. Conclusion

A production L3 KV-cache pool is valuable because it changes the reuse domain from one inference process to the cluster. The sound architecture is hierarchical: GPU L1 for latency, Host L2 for private capacity and registered transfer targets, and MoonCake Store DRAM for shared lifetime and scale. Correct reuse requires identity-compatible objects, a continuous prefix, rank agreement, sufficient expected compute savings, and a bounded recovery deadline. Writes should be asynchronous and deduplicated, with explicit backpressure so the mechanism that creates future hits does not harm current reads.

The supplied P2P and GDRDMA work provides important platform-validation methodology, but its strongest lesson is methodological: prove each layer independently and then prove the selected application path. For the current design, that path runs through Host L2. The final production decision must connect the entire chain - topology, memory registration, RDMA transfer, realized tokens, avoided prefill, TTFT, TPS, cold-path safety, and failure fallback. When that chain is measured and passes the stated gates, the L3 pool becomes a defensible system optimization rather than a cache-size claim.

Appendix A. Production checklist

Environment

- [ ] GPU model/count, driver, kernel, CUDA, BAR1, IOMMU, and P2P matrix recorded.
- [ ] `ibv_devices`, `ibv_devinfo`, and `rdma link show` match the intended NICs.
- [ ] GID, VLAN, MTU, routing, and congestion-control settings verified end to end.
- [ ] `ulimit -l`, container IPC/shared-memory, and pinned-memory limits are sufficient.

- [ ] GPU/NIC/CPU NUMA topology is documented and used for process placement.
- [ ] Each fabric independently passes transfer and integrity checks.

Cache and control plane

- [ ] All TP ranks use one explicit configuration source and identical cluster addresses.
- [ ] Model, tokenizer/template, dtype, page size, TP, and namespace fingerprints match.
- [ ] Inference instances advertise reachable RDMA addresses.
- [ ] Dedicated Store capacity leaves OS and operational headroom.
- [ ] Watermark eviction, health checks, restart behavior, and recovery objectives are defined.
- [ ] L3 failure returns to compute rather than failing inference.

Evidence

- [ ] Cold baseline retained.
- [ ] Same-instance and forced cross-instance warm tests pass.
- [ ] `storage_hit` is reconciled with `prefetched_tokens`.
- [ ] Threshold is calibrated around the measured break-even point.
- [ ] Read and write bandwidth/queues are separately visible.
- [ ] P50/P95/P99 TTFT, ITL, TPS, queue time, and GPU utilization are reported.
- [ ] Store loss, link loss, instance restart, and control-plane recovery are exercised.
- [ ] Long-duration run shows no new critical Xid, IOMMU, or provider errors.

Appendix B. Source traceability

REPORT TOPIC	SUPPLIED SOURCE BASIS
L1/L2/L3 roles, MoonCake control/data planes, paths, parameters, metrics, rollout	**M1**
Original cluster architecture and outcome mapping	**M2**
GPU P2P/GDRDMA capability chain, topology, validation, and reported substrate results	**M3**
Break-even model, path-truth distinction, evidence ladder, namespace contract	Synthesis in this report from M1-M3

Supplied materials

- **M1.** *agnes-2.5-flash Production-Grade L3 KV Cache Pool Technical Plan.* Supplied Markdown document, 2026.
- **M2.** *agnes-2.5-flash Production-Grade L3 KV Cache Pool Architecture.* Supplied SVG diagram, 2026.
- **M3.** *GPU Interconnect and GDRDMA Technical Proposal.* Supplied seven-page PDF, 2026.

The source materials mention SGLang implementation files for the 128-page storage batch, host-buffer registration, batch existence checks, and configuration precedence. Those links are treated here as part of M1's supplied evidence; the current report does not independently assert a repository commit or software-version pin.